

TechStar UjuziLab / UjuziHub

Complete System Architecture & Developer Blueprint

Full-Stack SRS: All 3 Phases — Market-Ready

Stack: React · Supabase · Vercel · Cloudflare

Version 1.0 — 2025

TABLE OF CONTENTS

1. Platform Vision & Overview
2. Tech Stack & Infrastructure
3. Database Schema — Every Table & Column
4. Authentication & Role System
5. Public Website — Pages & Components
6. Student Portal — All Flows & CRUDs
7. Instructor Portal — Course Builder & Analytics
8. Organization Portal
9. Moderator Portal
10. Super Admin Portal
11. Course System — Atomic Feature Spec
12. Video System
13. Quiz & Assessment System
14. Assignment System
15. Certification System
16. Payment & Enrollment System
17. Notification System
18. Community & Discussion System
19. Project & Innovation System
20. Search System
21. Analytics System
22. Moderation System
23. Email System
24. File & Media System
25. API Architecture — All Endpoints
26. Frontend Architecture & Folder Structure
27. State Management
28. Security & RLS Policies
29. Performance & Caching
30. Deployment & DevOps
31. Phase Roadmap — Phase 1, 2 & 3
32. UX/UI Design System
33. IoT & Device Integration (Phase 3)
34. AI Mentor Assistant (Phase 3)
35. Appendix — Environment Variables & Config

1. PLATFORM VISION & OVERVIEW

1.1 What We Are Building

TechStar UjuziLab (branded externally as UjuziHub) is Africa's modern learning and innovation ecosystem. It is NOT a simple LMS. It is a convergence of six systems:

- LMS — Courses, lessons, quizzes, assignments, certificates
- Creator Platform — Instructor tools, revenue, analytics
- Organization Platform — Schools, universities, companies, hubs
- Innovation Ecosystem — Projects, prototypes, labs, competitions
- Community Platform — Discussions, mentorship, hackathons
- Modern SaaS Application — Subscriptions, billing, analytics

1.2 Target Users

Students / Learners	Browse, enroll, learn, complete, get certified
Instructors / Creators	Create courses, upload content, earn revenue
Organization Admins	Manage institution, members, private programs
Moderators	Review content, handle reports, approve courses
Super Admins	Full platform control, analytics, configuration
Public Visitors	Browse catalog, view landing pages, sign up

1.3 Core Business Model

- Individual paid course purchases
- Free courses (lead generation)
- Organization licensing (bulk seats)
- Bootcamp enrollment fees
- Certification program fees
- Subscription plans (Phase 2)
- Sponsored learning programs
- Kit / hardware sales (Phase 2)
- Startup incubation fees (Phase 3)

2. TECH STACK & INFRASTRUCTURE

2.1 Frontend

Framework	React 18 + Next.js 14 (App Router)
Language	TypeScript (strict mode)
Styling	TailwindCSS + shadcn/ui components
Animation	Framer Motion
Data Fetching	TanStack Query (React Query v5)
State	Zustand
Forms	React Hook Form + Zod validation
Tables	TanStack Table v8
Charts	Recharts
Rich Text	Tiptap editor
Icons	Lucide React
Date/Time	date-fns

2.2 Backend (Supabase)

Database	Supabase PostgreSQL 15
Auth	Supabase Auth (JWT, OAuth)
Storage	Supabase Storage (media, docs)
Realtime	Supabase Realtime (notifications, chat)
Edge Functions	Supabase Edge Functions (Deno/TS)
RLS	Row Level Security on every table

2.3 External Services

Payments	Selcom API (Mobile Money, Cards, Bank)
Video CDN	Bunny.net (upload + streaming)
Email	Resend (transactional email)
Push	Firebase Cloud Messaging
Analytics	PostHog (product analytics)
Search (P2)	Typesense or Algolia
CDN	Cloudflare

2.4 Hosting & Deployment

Frontend	Vercel (automatic CI/CD from GitHub)
Database	Supabase hosted (managed Postgres)
Storage	Supabase Storage + Bunny.net CDN
DNS/CDN	Cloudflare (DDoS, caching, custom domain)
Monitoring	Vercel Analytics + PostHog
Error Tracking	Sentry (optional)

3. DATABASE SCHEMA — EVERY TABLE & COLUMN

All tables use UUID primary keys, created_at / updated_at timestamps, and Supabase RLS policies. Tables are grouped by domain.

3.1 Users & Auth Domain

Table: profiles

Extends Supabase auth.users. Created automatically via trigger on auth.users insert.

id	UUID PK — references auth.users(id)
username	TEXT UNIQUE NOT NULL
full_name	TEXT
bio	TEXT
avatar_url	TEXT
cover_url	TEXT
phone	TEXT
location	TEXT
website	TEXT
linkedin_url	TEXT
github_url	TEXT
date_of_birth	DATE
gender	TEXT — male female other prefer_not_to_say
language_preference	TEXT DEFAULT 'en'
timezone	TEXT DEFAULT 'Africa/Dar_es_Salaam'
is_instructor	BOOLEAN DEFAULT false
is_verified_instructor	BOOLEAN DEFAULT false
instructor_bio	TEXT
instructor_title	TEXT
onboarding_completed	BOOLEAN DEFAULT false
notification_preferences	JSONB DEFAULT '{}'
created_at	TIMESTAMPTZ DEFAULT now()
updated_at	TIMESTAMPTZ DEFAULT now()

Table: roles

id	UUID PK
name	TEXT UNIQUE — student instructor org_admin moderator super_admin
description	TEXT
permissions	JSONB
created_at	TIMESTAMPTZ

Table: user_roles

id	UUID PK
user_id	UUID FK → profiles(id)
role_id	UUID FK → roles(id)
granted_by	UUID FK → profiles(id) NULLABLE
granted_at	TIMESTAMPTZ DEFAULT now()
expires_at	TIMESTAMPTZ NULLABLE
is_active	BOOLEAN DEFAULT true

Table: instructor_applications

id	UUID PK
user_id	UUID FK → profiles(id)
expertise	TEXT[]
experience_years	INT
bio	TEXT
sample_content_url	TEXT
id_document_url	TEXT
status	TEXT — pending approved rejected
reviewed_by	UUID FK → profiles(id) NULLABLE
review_notes	TEXT
reviewed_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ

3.2 Organization Domain

Table: organizations

id	UUID PK
name	TEXT NOT NULL
slug	TEXT UNIQUE NOT NULL
description	TEXT
logo_url	TEXT
cover_url	TEXT
website	TEXT
email	TEXT
phone	TEXT
address	TEXT
city	TEXT
country	TEXT DEFAULT 'Tanzania'
type	TEXT — school university company ngo hub government community
size	TEXT — small medium large enterprise
is_verified	BOOLEAN DEFAULT false
is_active	BOOLEAN DEFAULT true
plan	TEXT — free starter professional enterprise
plan_expires_at	TIMESTAMPTZ
max_members	INT DEFAULT 50
max_courses	INT DEFAULT 10
settings	JSONB DEFAULT '{}'
created_by	UUID FK → profiles(id)
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: organization_members

id	UUID PK
org_id	UUID FK → organizations(id)
user_id	UUID FK → profiles(id)
role	TEXT — owner admin instructor student viewer
department	TEXT

invited_by	UUID FK → profiles(id) NULLABLE
joined_at	TIMESTAMPTZ DEFAULT now()
is_active	BOOLEAN DEFAULT true
UNIQUE	(org_id, user_id)

Table: organization_invitations

id	UUID PK
org_id	UUID FK → organizations(id)
email	TEXT NOT NULL
role	TEXT
token	TEXT UNIQUE
invited_by	UUID FK → profiles(id)
expires_at	TIMESTAMPTZ
accepted_at	TIMESTAMPTZ NULLABLE
status	TEXT — pending accepted expired cancelled
created_at	TIMESTAMPTZ

3.3 Course Domain

Table: categories

id	UUID PK
name	TEXT NOT NULL
slug	TEXT UNIQUE
description	TEXT
icon	TEXT
parent_id	UUID FK → categories(id) NULLABLE (for sub-categories)
sort_order	INT DEFAULT 0
is_active	BOOLEAN DEFAULT true
created_at	TIMESTAMPTZ

Table: tags

id	UUID PK
name	TEXT UNIQUE NOT NULL
slug	TEXT UNIQUE
usage_count	INT DEFAULT 0
created_at	TIMESTAMPTZ

Table: courses

id	UUID PK
title	TEXT NOT NULL
slug	TEXT UNIQUE NOT NULL
subtitle	TEXT
description	TEXT
thumbnail_url	TEXT
preview_video_url	TEXT
instructor_id	UUID FK → profiles(id)
org_id	UUID FK → organizations(id) NULLABLE
category_id	UUID FK → categories(id)
level	TEXT — beginner intermediate advanced all
language	TEXT DEFAULT 'en'

status	TEXT — draft pending_review published rejected archived
visibility	TEXT — public private org_only invite_only
price	DECIMAL(10,2) DEFAULT 0
currency	TEXT DEFAULT 'TZS'
discount_price	DECIMAL(10,2) NULLABLE
discount_expires_at	TIMESTAMPTZ NULLABLE
is_free	BOOLEAN DEFAULT false
duration_hours	DECIMAL(5,2)
total_lessons	INT DEFAULT 0
total_modules	INT DEFAULT 0
total_enrollments	INT DEFAULT 0
avg_rating	DECIMAL(3,2) DEFAULT 0
total_reviews	INT DEFAULT 0
completion_rate	DECIMAL(5,2) DEFAULT 0
what_you_learn	TEXT[]
requirements	TEXT[]
target_audience	TEXT
certificate_template_id	UUID NULLABLE
has_certificate	BOOLEAN DEFAULT true
is_featured	BOOLEAN DEFAULT false
is_bestseller	BOOLEAN DEFAULT false
published_at	TIMESTAMPTZ NULLABLE
rejected_reason	TEXT NULLABLE
reviewed_by	UUID FK → profiles(id) NULLABLE
seo_title	TEXT
seo_description	TEXT
search_vector	TSVECTOR (auto-updated trigger)
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: course_tags (junction)

course_id	UUID FK → courses(id)
tag_id	UUID FK → tags(id)
PRIMARY KEY	(course_id, tag_id)

Table: modules

id	UUID PK
course_id	UUID FK → courses(id) ON DELETE CASCADE
title	TEXT NOT NULL
description	TEXT
sort_order	INT NOT NULL
is_free_preview	BOOLEAN DEFAULT false
total_lessons	INT DEFAULT 0
duration_minutes	INT DEFAULT 0
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: lessons

id	UUID PK
module_id	UUID FK → modules(id) ON DELETE CASCADE
course_id	UUID FK → courses(id)
title	TEXT NOT NULL
slug	TEXT
type	TEXT — video article quiz assignment resource coding live
content	TEXT (markdown/rich text for articles)
video_url	TEXT (Bunny.net URL)
video_bunny_id	TEXT
video_duration_seconds	INT
resource_url	TEXT
resource_type	TEXT
sort_order	INT NOT NULL
duration_minutes	INT
is_free_preview	BOOLEAN DEFAULT false
is_published	BOOLEAN DEFAULT true
allow_download	BOOLEAN DEFAULT false
notes	TEXT
created_at	TIMESTAMPTZ

updated_at	TIMESTAMPTZ
-------------------	-------------

3.4 Quiz Domain

Table: quizzes

id	UUID PK
lesson_id	UUID FK → lessons(id) ON DELETE CASCADE
course_id	UUID FK → courses(id)
title	TEXT
description	TEXT
time_limit_minutes	INT NULLABLE (null = no limit)
pass_score	INT DEFAULT 70 (percentage)
max_attempts	INT DEFAULT 3 (null = unlimited)
shuffle_questions	BOOLEAN DEFAULT true
show_answers_after	TEXT — immediately after_pass after_deadline never
is_required_to_pass	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: questions

id	UUID PK
quiz_id	UUID FK → quizzes(id) ON DELETE CASCADE
type	TEXT — mcq true_false short_answer multi_select coding
question	TEXT NOT NULL
image_url	TEXT NULLABLE
explanation	TEXT (shown after answer)
points	INT DEFAULT 1
sort_order	INT
created_at	TIMESTAMPTZ

Table: question_options

id	UUID PK
question_id	UUID FK → questions(id) ON DELETE CASCADE
option_text	TEXT NOT NULL
is_correct	BOOLEAN DEFAULT false

sort_order	INT
-------------------	-----

Table: quiz_attempts

id	UUID PK
quiz_id	UUID FK → quizzes(id)
user_id	UUID FK → profiles(id)
enrollment_id	UUID FK → enrollments(id)
attempt_number	INT
score	DECIMAL(5,2)
max_score	INT
percentage	DECIMAL(5,2)
passed	BOOLEAN
started_at	TIMESTAMPTZ
submitted_at	TIMESTAMPTZ
time_taken_seconds	INT
answers	JSONB (array of {question_id, selected_option_ids, text_answer, is_correct, points_earned})

3.5 Assignment Domain

Table: assignments

id	UUID PK
lesson_id	UUID FK → lessons(id)
course_id	UUID FK → courses(id)
title	TEXT NOT NULL
description	TEXT
instructions	TEXT
allowed_file_types	TEXT[] — ['pdf','docx','zip','png','jpg']
max_file_size_mb	INT DEFAULT 10
max_files	INT DEFAULT 5
due_days_after_enrollment	INT NULLABLE
allow_late_submission	BOOLEAN DEFAULT false
grading_type	TEXT — auto manual peer
max_score	INT DEFAULT 100
pass_score	INT DEFAULT 50
rubric	JSONB NULLABLE
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: assignment_submissions

id	UUID PK
assignment_id	UUID FK → assignments(id)
user_id	UUID FK → profiles(id)
enrollment_id	UUID FK → enrollments(id)
content	TEXT
file_urls	TEXT[]
github_url	TEXT NULLABLE
status	TEXT — draft submitted grading graded revision_requested
score	INT NULLABLE
feedback	TEXT NULLABLE
graded_by	UUID FK → profiles(id) NULLABLE
graded_at	TIMESTAMPTZ NULLABLE

submitted_at	TIMESTAMPTZ
revision_count	INT DEFAULT 0
is_late	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

3.6 Enrollment & Progress Domain

Table: enrollments

id	UUID PK
user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id)
org_id	UUID FK → organizations(id) NULLABLE
enrolled_at	TIMESTAMPTZ DEFAULT now()
source	TEXT — purchase free org_assigned coupon admin
payment_id	UUID FK → payments(id) NULLABLE
coupon_id	UUID FK → coupons(id) NULLABLE
status	TEXT — active completed expired refunded cancelled
progress_percentage	DECIMAL(5,2) DEFAULT 0
completed_at	TIMESTAMPTZ NULLABLE
last_accessed_at	TIMESTAMPTZ
certificate_id	UUID FK → certificates(id) NULLABLE
expires_at	TIMESTAMPTZ NULLABLE
UNIQUE	(user_id, course_id)

Table: lesson_progress

id	UUID PK
enrollment_id	UUID FK → enrollments(id) ON DELETE CASCADE
lesson_id	UUID FK → lessons(id)
user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id)
status	TEXT — not_started in_progress completed
watch_time_seconds	INT DEFAULT 0
completion_percentage	DECIMAL(5,2) DEFAULT 0
completed_at	TIMESTAMPTZ NULLABLE
last_position_seconds	INT DEFAULT 0 (video resume)
notes	TEXT NULLABLE
UNIQUE	(enrollment_id, lesson_id)
updated_at	TIMESTAMPTZ

3.7 Payment Domain

Table: payments

id	UUID PK
user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id) NULLABLE
org_id	UUID FK → organizations(id) NULLABLE
amount	DECIMAL(10,2) NOT NULL
currency	TEXT DEFAULT 'TZS'
status	TEXT — pending processing completed failed refunded cancelled
payment_method	TEXT — mobile_money card bank_transfer
provider	TEXT DEFAULT 'selcom'
provider_ref	TEXT (Selcom transaction reference)
provider_order_id	TEXT
phone_number	TEXT NULLABLE (mobile money)
metadata	JSONB DEFAULT '{}'
receipt_url	TEXT NULLABLE
paid_at	TIMESTAMPTZ NULLABLE
expires_at	TIMESTAMPTZ (checkout session expiry)
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: coupons

id	UUID PK
code	TEXT UNIQUE NOT NULL
type	TEXT — percentage fixed
value	DECIMAL(10,2) NOT NULL
max_uses	INT NULLABLE
used_count	INT DEFAULT 0
course_id	UUID FK → courses(id) NULLABLE (null = all courses)
org_id	UUID FK → organizations(id) NULLABLE
valid_from	TIMESTAMPTZ
valid_until	TIMESTAMPTZ
is_active	BOOLEAN DEFAULT true

created_by	UUID FK → profiles(id)
created_at	TIMESTAMPTZ

Table: instructor_payouts

id	UUID PK
instructor_id	UUID FK → profiles(id)
amount	DECIMAL(10,2)
currency	TEXT DEFAULT 'TZS'
platform_fee	DECIMAL(10,2)
net_amount	DECIMAL(10,2)
status	TEXT — pending processing paid failed
payout_method	TEXT
payout_ref	TEXT
period_start	DATE
period_end	DATE
paid_at	TIMESTAMPTZ NULLABLE
created_at	TIMESTAMPTZ

3.8 Certificate Domain

Table: certificate_templates

id	UUID PK
name	TEXT
html_template	TEXT (Handlebars/Mustache template)
thumbnail_url	TEXT
is_default	BOOLEAN DEFAULT false
org_id	UUID FK → organizations(id) NULLABLE (org custom templates)
created_by	UUID FK → profiles(id)
created_at	TIMESTAMPTZ

Table: certificates

id	UUID PK
user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id)
enrollment_id	UUID FK → enrollments(id)
template_id	UUID FK → certificate_templates(id)
certificate_number	TEXT UNIQUE (e.g. UJZ-2025-00001)
verification_url	TEXT
pdf_url	TEXT
issued_at	TIMESTAMPTZ DEFAULT now()
metadata	JSONB (learner name, course, instructor, score, date)

3.9 Review & Rating Domain

Table: reviews

id	UUID PK
course_id	UUID FK → courses(id)
user_id	UUID FK → profiles(id)
enrollment_id	UUID FK → enrollments(id)
rating	INT CHECK (rating BETWEEN 1 AND 5)
title	TEXT
content	TEXT
is_verified_purchase	BOOLEAN DEFAULT true
helpful_count	INT DEFAULT 0
is_flagged	BOOLEAN DEFAULT false
is_published	BOOLEAN DEFAULT true
instructor_reply	TEXT NULLABLE
instructor_replied_at	TIMESTAMPTZ NULLABLE
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ
UNIQUE	(course_id, user_id)

3.10 Community Domain

Table: discussion_channels

id	UUID PK
name	TEXT NOT NULL
slug	TEXT UNIQUE
description	TEXT
type	TEXT — public course org private
course_id	UUID FK → courses(id) NULLABLE
org_id	UUID FK → organizations(id) NULLABLE
is_active	BOOLEAN DEFAULT true
sort_order	INT DEFAULT 0
created_by	UUID FK → profiles(id)
created_at	TIMESTAMPTZ

Table: posts

id	UUID PK
channel_id	UUID FK → discussion_channels(id)
author_id	UUID FK → profiles(id)
title	TEXT
content	TEXT NOT NULL
type	TEXT — question discussion announcement showcase resource
images	TEXT[]
upvote_count	INT DEFAULT 0
reply_count	INT DEFAULT 0
view_count	INT DEFAULT 0
is_pinned	BOOLEAN DEFAULT false
is_answered	BOOLEAN DEFAULT false
accepted_answer_id	UUID FK → comments(id) NULLABLE
is_flagged	BOOLEAN DEFAULT false
is_deleted	BOOLEAN DEFAULT false
tags	TEXT[]
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: comments

id	UUID PK
post_id	UUID FK → posts(id) ON DELETE CASCADE
author_id	UUID FK → profiles(id)
parent_id	UUID FK → comments(id) NULLABLE (for threading)
content	TEXT NOT NULL
upvote_count	INT DEFAULT 0
is_accepted_answer	BOOLEAN DEFAULT false
is_flagged	BOOLEAN DEFAULT false
is_deleted	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: post_votes

user_id	UUID FK → profiles(id)
post_id	UUID FK → posts(id)
vote_type	TEXT — up down
PRIMARY KEY	(user_id, post_id)

3.11 Project Domain

Table: projects

id	UUID PK
title	TEXT NOT NULL
slug	TEXT UNIQUE
description	TEXT
full_description	TEXT
creator_id	UUID FK → profiles(id)
org_id	UUID FK → organizations(id) NULLABLE
category	TEXT — agriculture education health environment energy smart_city other
tags	TEXT[]
status	TEXT — ideation prototyping testing completed published incubating
visibility	TEXT — public private org_only
thumbnail_url	TEXT
images	TEXT[]
video_url	TEXT NULLABLE
github_url	TEXT NULLABLE
demo_url	TEXT NULLABLE
hardware_components	TEXT[]
technologies	TEXT[]
team_size	INT
view_count	INT DEFAULT 0
like_count	INT DEFAULT 0
is_featured	BOOLEAN DEFAULT false
is_competition_entry	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: project_team_members

project_id	UUID FK → projects(id)
user_id	UUID FK → profiles(id)
role	TEXT

joined_at	TIMESTAMPTZ
PRIMARY KEY	(project_id, user_id)

3.12 Notification Domain

Table: notifications

id	UUID PK
user_id	UUID FK → profiles(id)
type	TEXT — enrollment quiz_result assignment_feedback certificate new_reply mention payment announcement course_update
title	TEXT
body	TEXT
data	JSONB (action URL, entity IDs)
is_read	BOOLEAN DEFAULT false
read_at	TIMESTAMPTZ NULLABLE
created_at	TIMESTAMPTZ

Table: notification_preferences

user_id	UUID PK FK → profiles(id)
email_enrollment	BOOLEAN DEFAULT true
email_quiz	BOOLEAN DEFAULT true
email_certificate	BOOLEAN DEFAULT true
email_announcements	BOOLEAN DEFAULT true
email_marketing	BOOLEAN DEFAULT false
push_enabled	BOOLEAN DEFAULT true
push_reminders	BOOLEAN DEFAULT true
in_app_all	BOOLEAN DEFAULT true
updated_at	TIMESTAMPTZ

3.13 Content Moderation Domain

Table: content_reports

id	UUID PK
reporter_id	UUID FK → profiles(id)
content_type	TEXT — course post comment project profile
content_id	UUID NOT NULL
reason	TEXT — spam inappropriate copyright misleading hate_speech other
description	TEXT
status	TEXT — pending reviewing resolved dismissed
resolved_by	UUID FK → profiles(id) NULLABLE
resolution_notes	TEXT NULLABLE
created_at	TIMESTAMPTZ
resolved_at	TIMESTAMPTZ NULLABLE

Table: audit_logs

id	UUID PK
actor_id	UUID FK → profiles(id) NULLABLE
action	TEXT — user.created course.published payment.completed ...
entity_type	TEXT
entity_id	UUID
old_data	JSONB NULLABLE
new_data	JSONB NULLABLE
ip_address	INET
user_agent	TEXT
created_at	TIMESTAMPTZ DEFAULT now()

3.14 Events & Programs Domain

Table: programs

id	UUID PK
title	TEXT NOT NULL
slug	TEXT UNIQUE
description	TEXT
type	TEXT — bootcamp workshop hackathon competition webinar certification_program
thumbnail_url	TEXT
banner_url	TEXT
org_id	UUID FK → organizations(id) NULLABLE
instructor_id	UUID FK → profiles(id) NULLABLE
price	DECIMAL(10,2) DEFAULT 0
max_participants	INT NULLABLE
registration_deadline	TIMESTAMPTZ
start_date	TIMESTAMPTZ
end_date	TIMESTAMPTZ
format	TEXT — online physical hybrid
location	TEXT NULLABLE
status	TEXT — draft open closed ongoing completed cancelled
certificate_offered	BOOLEAN DEFAULT true
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

Table: program_registrations

id	UUID PK
program_id	UUID FK → programs(id)
user_id	UUID FK → profiles(id)
status	TEXT — pending confirmed attended completed cancelled
payment_id	UUID FK → payments(id) NULLABLE
registered_at	TIMESTAMPTZ
UNIQUE	(program_id, user_id)

3.15 Lab Resources Domain

Table: lab_resources

id	UUID PK
title	TEXT NOT NULL
description	TEXT
type	TEXT — circuit_diagram code_repo documentation 3d_design ai_model iot_simulation>manual data set
category	TEXT — arduino esp32 raspberry_pi ai iot robotics pcb other
file_url	TEXT
external_url	TEXT
thumbnail_url	TEXT
tags	TEXT[]
is_public	BOOLEAN DEFAULT true
download_count	INT DEFAULT 0
uploaded_by	UUID FK → profiles(id)
created_at	TIMESTAMPTZ

3.16 Wishlists, Bookmarks & Social

Table: wishlists

user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id)
added_at	TIMESTAMPTZ
PRIMARY KEY	(user_id, course_id)

Table: follows

follower_id	UUID FK → profiles(id)
following_id	UUID FK → profiles(id)
created_at	TIMESTAMPTZ
PRIMARY KEY	(follower_id, following_id)

Table: cart_items

id	UUID PK
user_id	UUID FK → profiles(id)
course_id	UUID FK → courses(id)
added_at	TIMESTAMPTZ
coupon_code	TEXT NULLABLE
UNIQUE	(user_id, course_id)

4. AUTHENTICATION & ROLE SYSTEM

4.1 Auth Provider — Supabase Auth

Supabase handles all authentication. On signup, a database trigger automatically creates a profiles row.

4.2 Auth Flows & Screens

Flow	Steps	Supabase Method
Email Signup	Enter email + password + name → Verify email → Create profile → Redirect to onboarding	supabase.auth.signUp()
Email Login	Enter credentials → JWT issued → Redirect to dashboard	supabase.auth.signInWithPassword()
Google OAuth	Click Google → OAuth redirect → Profile auto-created → Dashboard	supabase.auth.signInWithOAuth({ provider: 'google' })
Forgot Password	Enter email → Receive reset link → Click link → Enter new password	supabase.auth.resetPasswordForEmail()
Reset Password	Token in URL → New password form → supabase.auth.updateUser()	supabase.auth.updateUser({ password })
Email Verification	Link in email → Confirm token → Redirect	handled by Supabase redirect
Logout	Clear session → Redirect to home	supabase.auth.signOut()
Session Refresh	Auto-refresh JWT before expiry	supabase.auth.onAuthStateChange()

4.3 User Roles & Permissions Matrix

Permission	Student	Instructor	Org Admin	Moderator	Super Admin
Enroll in courses	✓	✓	✓	✓	✓
Create courses	✗	✓	✓	✗	✓
Manage own courses	✗	✓	✓	✗	✓
View own analytics	✓	✓	✓	✓	✓
Manage org members	✗	✗	✓	✗	✓

Review/moderate content	X	X	X	✓	✓
Approve/reject courses	X	X	X	✓	✓
Manage all users	X	X	X	X	✓
Manage payments	X	X	X	X	✓
Platform configuration	X	X	X	X	✓
Issue refunds	X	X	X	X	✓
View all analytics	X	X	X	X	✓

4.4 Onboarding Flow

1. User signs up with email/Google
2. Email verification (email signup only)
3. Onboarding screen: Select role — I want to Learn / I want to Teach / Both
4. Fill profile: name, avatar, location, interests
5. If instructor selected: fill instructor application form
6. Redirect to appropriate dashboard

5. PUBLIC WEBSITE — PAGES & COMPONENTS

5.1 Homepage (/)

Sections:

- Hero Section: Headline, subheadline, CTA buttons (Browse Courses / Start Teaching), hero image/animation, stats bar (X learners, Y courses, Z instructors)
- Featured Courses: Horizontal scroll cards with title, instructor, rating, price, thumbnail
- Categories Grid: Icon cards for Python, AI, Robotics, IoT, Web Dev, etc.
- How It Works: 3-step process with icons
- Stats Banner: Total learners, courses, certificates issued, organizations
- Innovation Highlights: Featured projects/innovations from learners
- Organizations Section: Partner logos grid
- Upcoming Programs: Bootcamp/event cards
- Testimonials: Learner quote cards
- CTA Banner: Join the ecosystem
- Footer: Links, social media, contact, newsletter

5.2 Courses Page (/courses)

- Search bar with instant results
- Filter sidebar: Category, Level, Price (Free/Paid/Price range), Duration, Language, Rating
- Sort options: Newest, Most Popular, Highest Rated, Price (Low/High)
- Course cards grid (24 per page) with pagination or infinite scroll
- Active filter chips with remove option
- Result count display

5.3 Course Detail Page (/courses/[slug])

- Sticky sidebar: Price, discount countdown, Enroll/Buy CTA, course stats
- Hero: Thumbnail/preview video, title, subtitle, rating, enrollment count, last updated
- What You'll Learn: Checkbox grid
- Course Curriculum: Accordion modules/lessons, free preview indicators, total duration
- Instructor section: Avatar, bio, stats (courses, students, rating)
- Reviews section: Rating breakdown chart, paginated reviews, helpful votes
- Related Courses: Horizontal scroll
- Share buttons, Add to Wishlist button
- Mobile: Bottom sticky CTA bar

5.4 Remaining Public Pages

/programs	Bootcamps/events list with filter by type, date, format
/organizations	Partner org grid with search, type filter, organization cards
/projects	Public project showcase grid with category/tag filters
/community	Preview of community discussions, invite to join
/pricing	Plan comparison table, FAQ
/about	Mission, team, timeline, values
/contact	Contact form + location map
/verify-certificate	Certificate verification by code
/blog	Phase 2 — articles, tutorials
/auth/login	Login form with Google OAuth
/auth/register	Signup form with Google OAuth
/auth/forgot-password	Password reset request
/auth/reset-password	New password form (token in URL)

6. STUDENT PORTAL — ALL FLOWS & CRUDs

6.1 Student Dashboard (/dashboard)

- Continue Learning widget: Last accessed course, progress bar, Resume button
- My Stats: Courses enrolled, completed, certificates earned, hours learned
- Learning streak calendar (GitHub-style heatmap)
- Upcoming deadlines: Assignments due
- Recently earned certificates
- Recommended courses (based on enrolled/completed categories)
- Announcements from instructors/platform

6.2 My Courses (/dashboard/my-courses)

- Tabs: All | Active | Completed | Archived
- Course cards with: thumbnail, title, progress bar %, last accessed
- Search and filter by category, status
- Sort by: Last accessed, Enrolled date, Progress
- Click → goes to course player

6.3 Course Player (/learn/[courseSlug]/[lessonSlug])

- Left sidebar: Collapsible curriculum tree (modules > lessons) with completion checkmarks
- Main content area:
 - Video player (Bunny.net) with: speed control, captions, fullscreen, volume, resume from last position, mark complete button
 - Article viewer: rendered markdown/rich text
 - Quiz interface: question-by-question or all at once, timer if set
 - Assignment submission: file upload, text editor, submit button
- Bottom navigation: Previous / Next lesson buttons
- Right panel (collapsible): Lesson notes, Q&A tab, Resources tab
- Progress auto-saved on each lesson completion
- Keyboard shortcuts: Space=play/pause, F=fullscreen, Arrow keys=seek

6.4 Quizzes — Student Flow

7. Student arrives at quiz lesson
8. Quiz intro screen: rules, time limit, attempts remaining
9. Start Quiz button
10. Question navigation: one at a time or paginated
11. Timer countdown (if enabled)

12. Submit button (confirm dialog)
13. Instant results: Score, pass/fail, per-question breakdown
14. If passed → lesson marked complete, move to next
15. If failed → retry button (if attempts remain) or locked
16. Explanations shown per answer (based on settings)

6.5 Assignments — Student Flow

17. Arrive at assignment lesson — read instructions, rubric
18. Upload files (drag & drop, multi-file) and/or write in rich text editor
19. Optionally add GitHub repo link
20. Save as draft or Submit
21. Status tracker: Draft → Submitted → Grading → Graded / Revision Requested
22. Receive grade + feedback notification
23. If revision requested: edit and resubmit

6.6 Certificates (/dashboard/certificates)

- Grid of earned certificates with course name, issue date, certificate number
- Click to view full certificate (modal or full page)
- Download as PDF
- Share link (public verification URL)
- LinkedIn share button

6.7 Wishlist (/dashboard/wishlist)

- Grid of wishlisted courses
- Remove from wishlist
- Enroll/Purchase directly from wishlist

6.8 Cart & Checkout (/cart)

- Cart items list with price, discount badge
- Coupon code input with apply/remove
- Order summary: subtotal, discount, total
- Checkout button → Payment page
- Payment page: Mobile Money input (phone number), or Card details
- Selcom payment initiated → redirect / awaiting confirmation
- Success page: enrollment confirmed, go to course
- Failure page: retry option

6.9 Profile & Settings

/profile/[username]	Public profile: avatar, bio, enrolled courses, certificates, projects
/dashboard/settings/profile	Edit: name, bio, avatar, cover, location, social links
/dashboard/settings/account	Change email, change password, delete account
/dashboard/settings/notifications	Toggle each notification type (email, push, in-app)
/dashboard/settings/billing	Payment history, receipts download, saved payment methods
/dashboard/settings/privacy	Account visibility, data export

7. INSTRUCTOR PORTAL — COURSE BUILDER & ANALYTICS

7.1 Instructor Dashboard

- Revenue widget: This month, last month, total (line chart)
- Top performing courses: enrollment and completion rates
- Recent reviews (star + comment)
- New enrollments this week
- Student activity heatmap
- Quick actions: Create New Course, View Analytics, Withdraw Earnings

7.2 Course Builder — Full Workflow

URL: /instructor/courses/[courseId]/edit

Step 1 — Basic Info

- Title, subtitle, description (rich text with Tiptap)
- Category, sub-category, level, language
- Tags (auto-complete from existing tags)
- Thumbnail upload (drag & drop, crop tool, Supabase Storage)
- Preview video upload or YouTube/Bunny.net URL
- Auto-save every 30 seconds

Step 2 — Curriculum Builder

- Add Module button → inline editable module card
- Drag-and-drop reorder modules (dnd-kit)
- Within each module: Add Lesson dropdown (Video / Article / Quiz / Assignment / Resource)
- Drag-and-drop reorder lessons within/across modules
- Lesson editing inline or in modal

Lesson Editor — Video

- Upload video file (chunked upload to Bunny.net via Edge Function)
- Upload progress bar with cancel option
- Auto-thumbnail extraction from video
- Attach downloadable resources (PDF, ZIP, etc.)
- Add lesson notes (markdown)
- Toggle: Free preview, Allow download

Lesson Editor — Article

- Full Tiptap rich text editor: headings, bold, italic, lists, links, images, code blocks, embeds
- Estimated read time auto-calculated
- Attach resources

Lesson Editor — Quiz

- Add Question button → choose type (MCQ, T/F, Short Answer, Multi-select)
- For MCQ/Multi-select: Add options, mark correct answer(s), add explanation
- Per-question points assignment
- Quiz settings: time limit, pass score, attempts, shuffle, show answers
- Drag-to-reorder questions

Lesson Editor — Assignment

- Rich text instructions + rubric builder
- Allowed file types multi-select
- Due days after enrollment, max files, max score

Step 3 — Requirements & Objectives

- What students will learn: up to 8 bullet points (required)
- Course requirements/prerequisites
- Target audience description

Step 4 — Pricing

- Free or Paid toggle
- Price input (TZS) with USD equivalent preview
- Discount price + expiry date
- Visibility: Public / Organization Only / Invite Only
- Coupon management (create, list, deactivate)

Step 5 — SEO & Certificates

- SEO title, SEO description
- Certificate toggle: Enable/Disable
- Select certificate template

Step 6 — Review & Submit

- Completeness checklist (all required fields validated)
- Preview mode: see the course as a student would
- Submit for Review button

- Status badge: Draft / Pending Review / Published / Rejected

7.3 Instructor Analytics (/instructor/analytics)

- Revenue: Total, per course, per month (line + bar chart)
- Enrollments: Total, per course, per day chart
- Completion rates: Per course funnel chart
- Student engagement: Average watch time, drop-off points per lesson
- Reviews: Rating distribution, recent reviews
- Top lessons by completion, top lessons by drop-off

7.4 Instructor Earnings & Payouts (/instructor/earnings)

- Balance: Available for payout, pending, paid out
- Transaction log: Per-enrollment earnings with course name, date, amount
- Platform fee shown (e.g. 30%)
- Request payout: Enter amount + payout method (Mobile Money/Bank)
- Payout history table

7.5 Instructor Students (/instructor/students)

- Table: Student name, course, enrolled date, progress %, last active
- Filter by course
- Click student → view detailed progress per lesson, quiz scores, assignments
- Send message to student (future direct message)

8. ORGANIZATION PORTAL

8.1 Organization Dashboard

- Member count, active learners, course completions this month
- Top courses by completion within org
- Upcoming programs/events
- Recent activity feed

8.2 Organization Profile Management

- Edit: name, description, logo, cover, website, contact
- Custom branding (logo on certificates)
- Organization type and verification status

8.3 Member Management (/org/[slug]/members)

- Members table: name, role, department, joined date, status
- Invite by email: enter email, select role → invitation email sent
- Bulk invite via CSV upload
- Change member role (admin, instructor, student)
- Remove member
- Pending invitations list with resend/cancel

8.4 Organization Courses

- Create private org-only courses (same course builder as instructor)
- Assign existing public courses to org
- Assign courses to specific members or all members
- Set enrollment deadlines

8.5 Organization Analytics

- Overall completion rates per course
- Member engagement table: individual progress
- Export report as CSV/PDF
- Department-level breakdown

8.6 Programs & Events

- Create and manage bootcamps, workshops, competitions for the org
- Registration management, attendance tracking

9. MODERATOR PORTAL

9.1 Course Review Queue

- Table of courses in pending_review status
- Columns: Title, Instructor, Category, Submitted date, Lesson count
- Click → full course preview (all content visible to moderator)
- Approve button → course status → published, instructor notified
- Reject button → modal to write rejection reason → instructor notified
- Request changes button → notes sent to instructor

9.2 Content Reports

- Reports queue: content_type, reason, reporter, date
- View reported content inline
- Actions: Dismiss, Warn User, Remove Content, Ban User
- Resolution notes field

9.3 Community Moderation

- Flagged posts and comments queue
- Actions: approve, remove, warn author, pin post

10. SUPER ADMIN PORTAL

10.1 Admin Dashboard (/admin)

- Platform KPIs: Total users, active today, total revenue MTD, total courses, enrollments
- Revenue chart: daily/weekly/monthly/yearly
- User growth chart
- Top courses by revenue and enrollment
- Recent payments table
- System health indicators

10.2 User Management (/admin/users)

- Full users table with advanced filters: role, status, date joined, org
- Search by name/email
- View user detail: profile, roles, enrollments, payments, activity
- Edit user: change role, suspend/unsuspend account, force password reset
- Delete user (with data anonymization)
- Impersonate user (view platform as that user)

10.3 Course Management (/admin/courses)

- All courses table: status, instructor, category, price, enrollments
- Filter by status, category, instructor
- Force publish / unpublish / archive
- Feature/unfeature, mark bestseller
- Edit any course field (admin override)

10.4 Organization Management (/admin/organizations)

- All organizations table: name, type, plan, member count, verified status
- Create organization manually
- Edit org details, change plan, set limits
- Verify / unverify organization
- View org members and activity

10.5 Payment Management (/admin/payments)

- All transactions table: user, course, amount, method, status, date
- Filter by status, method, date range

- Issue refund (manual): update payment status to refunded, revoke enrollment
- View Selcom webhook logs
- Revenue reports: daily, monthly, by course, by instructor
- Export CSV

10.6 Category & Tag Management (/admin/settings/categories)

- CRUD categories and sub-categories
- Reorder categories (drag-drop)
- Merge tags

10.7 Platform Configuration (/admin/settings)

- Platform name, logo, colors, footer links
- Instructor revenue split % (default 70%)
- Selcom API keys and webhook secret
- Bunny.net API key and library ID
- Email provider API key
- Feature flags: enable/disable modules (community, projects, IoT lab)
- Certificate number prefix and format
- Maintenance mode toggle

10.8 Instructor Applications (/admin/instructor-applications)

- Applications table: applicant, expertise, submitted date, status
- Review: view profile, application, sample content
- Approve → grant instructor role → send welcome email
- Reject → send rejection with reason

11. COURSE SYSTEM — ATOMIC FEATURE SPEC

11.1 Course States & Transitions

draft → pending_review	Instructor submits for review
pending_review → published	Moderator/Admin approves
pending_review → rejected	Moderator rejects with reason
rejected → draft	Instructor edits and resubmits
published → archived	Instructor or admin archives
archived → draft	Instructor restores to draft
published → draft	Instructor unpublishes (edit mode)

11.2 Course Progress Calculation

Progress % = (completed lessons / total published lessons) * 100

A lesson is 'completed' when:

- Video: watched $\geq 85\%$ of duration
- Article: scrolled to bottom and marked complete
- Quiz: passed (score $\geq$ pass_score) OR attempted and submitted
- Assignment: submitted (not necessarily graded)
- Resource: downloaded or marked complete manually

11.3 Certificate Trigger

Certificate is automatically generated (via Edge Function) when:

- enrollment.progress_percentage reaches 100%
- All required quizzes are passed
- All required assignments are submitted
- course.has_certificate === true

12. VIDEO SYSTEM

12.1 Video Upload Flow

24. Instructor selects video file in lesson editor
25. Client creates upload ticket via Supabase Edge Function → Bunny.net API
26. Client uploads directly to Bunny.net using TUS protocol (resumable)
27. Upload progress tracked client-side with progress bar
28. Bunny.net processes video (transcodes to multiple qualities)
29. Webhook from Bunny.net → Edge Function → update lesson.video_bunny_id + video_url
30. Lesson marked as video_ready

12.2 Video Player Features

Streaming	HLS adaptive bitrate via Bunny.net CDN
Resume	last_position_seconds saved to lesson_progress on pause/exit
Speed control	0.5x, 0.75x, 1x, 1.25x, 1.5x, 2x
Captions	Upload .srt/.vtt files per lesson
Quality	Auto / 1080p / 720p / 480p / 360p
Fullscreen	Native browser fullscreen API
Keyboard	Space=play, F=fullscreen, M=mute, ←/→=seek 10s
Progress tracking	Watch event fires every 10s → update watch_time_seconds
Completion	Fires at 85% watch → mark lesson complete
Theater mode	Expand video, collapse sidebar

13. QUIZ & ASSESSMENT SYSTEM

13.1 Quiz Engine — Frontend State Machine

- State: idle → intro → active → submitted → result
- Timer: useEffect countdown, auto-submit on 0
- Question navigation: sidebar index with status dots (unanswered/answered/flagged)
- Flag question feature (skip and come back)
- Shuffle questions client-side on start (Fisher-Yates)
- Offline tolerance: answers stored in localStorage as backup

13.2 Grading Logic (Edge Function)

- MCQ: compare selected_option_id to question_options.is_correct
- Multi-select: all correct options selected + no incorrect = full points
- True/False: same as MCQ
- Short answer: stored, requires manual review (flagged for instructor)
- Total score = sum of earned points / sum of max points * 100
- passed = percentage >= quiz.pass_score

14. ASSIGNMENT SYSTEM

14.1 File Upload Flow

31. Student selects files (drag & drop with react-dropzone)
32. Client validates type and size
33. Upload to Supabase Storage: assignments/{submission_id}/{filename}
34. Signed URLs generated for secure access
35. Submission record created/updated in database

14.2 Grading Interface (Instructor)

- Navigate to student submission from Students tab
- View submitted files (inline PDF viewer, image viewer, code viewer)
- Score input (0 to max_score)
- Feedback rich text editor
- Request Revision button (sends notification, sets status to revision_requested)
- Save Grade button → notification sent to student

15. CERTIFICATION SYSTEM

15.1 Certificate Generation (Edge Function)

36. Trigger: enrollment progress reaches 100%
37. Generate certificate number: UJZ-{YEAR}-{SEQUENCE:05d}
38. Load HTML template from certificate_templates
39. Inject: learner name, course title, instructor name, completion date, certificate number, verification URL
40. Render HTML to PDF using puppeteer/headless Chrome
41. Upload PDF to Supabase Storage: certificates/{user_id}/{cert_id}.pdf
42. Insert certificate record to database
43. Update enrollment.certificate_id
44. Send email with certificate attached and share link

15.2 Certificate Verification

Public page at /verify-certificate → enter certificate number → displays:

- Learner name, course title, instructor, issue date
- Platform logo and digital seal
- Valid / Invalid status

16. PAYMENT & ENROLLMENT SYSTEM

16.1 Selcom Payment Flow (Full)

45. User clicks Enroll/Buy → cart → checkout
46. Frontend calls POST /api/payments/initiate (Edge Function)
47. Edge Function creates payment record (status: pending)
48. Edge Function calls Selcom API: Create Order with amount, currency, callback URL
49. Selcom returns order_id and checkout_url
50. Update payment with provider_order_id
51. Frontend redirects user to Selcom checkout URL
52. User pays via Mobile Money/Card on Selcom page
53. Selcom sends webhook to POST /api/payments/webhook (Edge Function)
54. Edge Function: verify webhook signature (HMAC), update payment status
55. If completed: create enrollment, send confirmation email, trigger notification
56. Frontend polls GET /api/payments/{id}/status every 3s (or via Supabase Realtime)
57. Show success/failure page

16.2 Free Enrollment

58. User clicks Enroll Free on course page
59. POST /api/enrollments/create (if authenticated)
60. Create enrollment record (source: free)
61. Redirect to course player

16.3 Revenue Split

On payment completion Edge Function:

- $\text{instructor_earning} = \text{payment.amount} * (1 - \text{platform_fee_rate})$
- $\text{platform_earning} = \text{payment.amount} * \text{platform_fee_rate}$
- Both logged to transactions table
- instructor_balance updated

17. NOTIFICATION SYSTEM

17.1 Notification Types & Triggers

enrollment	Student enrolled in a course
quiz_result	Quiz submitted and graded
assignment_feedback	Instructor graded an assignment
certificate	Certificate generated
course_update	Instructor published new lesson/announcement
new_reply	Someone replied to your post/comment
mention	@mention in community
payment_success	Payment completed
payment_failed	Payment failed
course_approved	Moderator approved course
course_rejected	Moderator rejected course with reason
new_enrollment	New student enrolled (to instructor)
new_review	New review on your course

17.2 In-App Notification Bell

- Bell icon in topbar with unread count badge
- Dropdown: last 10 notifications, each with icon, title, time ago, read indicator
- Mark all as read
- View all notifications page
- Supabase Realtime subscription on notifications table for real-time updates

17.3 Email Notifications (Resend)

- Welcome email on signup (HTML template)
- Enrollment confirmation with course details
- Certificate email with PDF attachment and share link
- Payment receipt with PDF invoice
- Password reset email
- Organization invitation email with join link
- All emails use branded HTML templates with platform logo

18. COMMUNITY & DISCUSSION SYSTEM

18.1 Channels

- General channels: #general, #announcements, #showcase, #help
- Per-course Q&A channel (auto-created on course publish)
- Per-organization channels
- User can subscribe/unsubscribe channels

18.2 Posts & Interactions

- Create post: type selection, title, rich text content, image uploads, tags
- Upvote posts (toggle)
- Reply to posts (nested comments, 2 levels max)
- Accept answer (original poster can mark a reply as accepted answer)
- @mention users with autocomplete (triggers mention notification)
- Edit/delete own posts and comments
- Pin posts (moderator/instructor)
- Share post URL
- Flag content for moderation

19. PROJECT & INNOVATION SYSTEM

19.1 Project Creation Flow

62. Click 'Submit Project' from Projects section
63. Step 1: Basic Info — title, description, category, tags
64. Step 2: Team — add team members by username
65. Step 3: Technical — technologies, hardware components, GitHub URL, demo URL
66. Step 4: Media — upload thumbnail, additional images, demo video URL
67. Step 5: Publish — visibility, competition entry toggle
68. Project goes live as public showcase

19.2 Project Detail Page

- Hero: thumbnail, title, creator, tech stack badges, status
- Tabs: Overview | Team | Gallery | Documentation | Updates
- Like button, share, follow creator
- Comments section
- Related projects

20. SEARCH SYSTEM

20.1 Implementation

- PostgreSQL full-text search using tsvector on courses(title, description, instructor name)
- search_vector column auto-updated via trigger
- Query: to_tsquery with ranking (ts_rank)
- Unified search endpoint: returns courses + instructors + projects + organizations
- Search suggestions (debounced 300ms, min 2 chars)
- Recent searches stored in localStorage
- Phase 2: Migrate to Typesense for typo-tolerance and faceted search

21. ANALYTICS SYSTEM

21.1 PostHog Integration

- Track events: page_view, course_viewed, course_enrolled, lesson_started, lesson_completed, quiz_attempted, certificate_earned, payment_initiated, payment_completed
- User identification after login: posthog.identify(userId, { name, email, role })
- Feature flags via PostHog for A/B testing

21.2 Custom Analytics (Admin Dashboard)

- Revenue: SUM(amount) GROUP BY day/month with pg queries
- Enrollments: COUNT per course, per day
- Completion rates: AVG(progress_percentage) per course
- Watch time: SUM(watch_time_seconds) per user/course
- All served via Supabase RPC functions (stored procedures) for performance

22. MODERATION SYSTEM

22.1 Automated Checks

- Basic profanity filter on post/comment creation (Edge Function)
- Duplicate course title detection
- Copyright flag: check if same video ID reused

22.2 Manual Review

- Reports processed by moderators in queue
- Content hidden immediately if 5+ reports received (pending review)
- Ban system: temporary (n days) or permanent
- Ban logged to audit_logs

23. EMAIL SYSTEM

Use Resend with React Email templates. All Edge Functions use Resend SDK.

welcome	On signup — welcome + verification link
verify-email	Email verification
reset-password	Password reset link
enrollment-confirmation	Course name, instructor, start learning CTA
payment-receipt	Amount, course, transaction ID, PDF invoice attached
certificate-issued	Congrats, PDF attached, share link, LinkedIn CTA
course-approved	To instructor — course is live
course-rejected	To instructor — reason + edit link
assignment-graded	Score, feedback, link to submission
org-invitation	Join {org} — accept link (expires 7 days)
new-review	To instructor — reviewer name, rating, comment
weekly-digest	Phase 2 — learner progress summary

24. FILE & MEDIA SYSTEM

24.1 Supabase Storage Buckets

avatars	Profile pictures — public bucket — 2MB max
covers	Profile/org cover images — public — 5MB max
thumbnails	Course thumbnails — public — 3MB max
assignments	Assignment submissions — private — 50MB max
resources	Lesson downloadable resources — RLS by enrollment
certificates	Generated certificate PDFs — RLS by owner
org-assets	Org logos and media — public
lab-resources	Lab resource files — public or RLS

24.2 File Upload Pattern (Frontend)

```
const { data, error } = await supabase.storage
  .from('thumbnails')
  .upload(`courses/${courseId}/thumb.jpg`, file, {
    upsert: true, contentType: file.type
  });
const { data: { publicUrl } } =
  supabase.storage.from('thumbnails').getPublicUrl(data.path);
```

25. API ARCHITECTURE — ALL ENDPOINTS

All non-CRUD operations use Supabase Edge Functions (Deno TypeScript). Standard CRUD uses Supabase client directly from frontend with RLS.

25.1 Edge Functions

POST /functions/v1/payments/initiate	Create Selcom order, return checkout URL
POST /functions/v1/payments/webhook	Verify Selcom webhook, process enrollment
GET /functions/v1/payments/:id/status	Poll payment status
POST /functions/v1/enrollments/create	Create free enrollment, trigger notifications
POST /functions/v1/certificates/generate	Render HTML to PDF, upload, record
POST /functions/v1/videos/upload-ticket	Create Bunny.net upload ticket
POST /functions/v1/videos/webhook	Handle Bunny.net encoding completion
POST /functions/v1/notifications/send	Send push + email notification
POST /functions/v1/search	Full-text search across entities
GET /functions/v1/analytics/instructor	Instructor revenue + enrollment stats
GET /functions/v1/analytics/admin	Platform-wide KPI aggregates
POST /functions/v1/orgs/invite	Send org invitation email + create invitation record
POST /functions/v1/quizzes/grade	Grade quiz attempt, check certificate trigger
POST /functions/v1/courses/publish	Validate completeness + submit for review
POST /functions/v1/auth/onboard	Set role, create profile, welcome email

25.2 Frontend Supabase Queries (examples)

Fetch published courses with filter

```
supabase.from('courses').select(`
  *, instructor:profiles(username, full_name, avatar_url),
  category:categories(name, slug)
`).eq('status', 'published').eq('visibility', 'public')
```

```
.order('created_at', { ascending: false }).range(0, 23)
```

Student enrollment check

```
supabase.from('enrollments').select('id, progress_percentage')  
.eq('user_id', userId).eq('course_id', courseId).single()
```

Real-time notifications

```
supabase.channel('notifications')  
  .on('postgres_changes', { event: 'INSERT', schema: 'public',  
    table: 'notifications', filter: `user_id=eq.${userId}` },  
    payload => addNotification(payload.new))  
  .subscribe()
```

26. FRONTEND ARCHITECTURE & FOLDER STRUCTURE

```

src/
├── app/                                # Next.js App Router
│   ├── (public)/                      # No auth required
│   │   ├── page.tsx                  # Homepage
│   │   ├── courses/
│   │   │   ├── page.tsx              # Course catalog
│   │   │   └── [slug]/page.tsx      # Course detail
│   │   ├── programs/page.tsx
│   │   ├── organizations/page.tsx
│   │   ├── projects/page.tsx
│   │   ├── verify-certificate/page.tsx
│   │   └── auth/
│   │       ├── login/page.tsx
│   │       ├── register/page.tsx
│   │       ├── forgot-password/page.tsx
│   │       └── reset-password/page.tsx
│   ├── (dashboard)/                  # Student portal
│   │   ├── layout.tsx                # Sidebar + topbar
│   │   ├── dashboard/page.tsx
│   │   ├── my-courses/page.tsx
│   │   ├── certificates/page.tsx
│   │   ├── wishlist/page.tsx
│   │   ├── projects/page.tsx
│   │   ├── community/
│   │   │   ├── page.tsx
│   │   │   └── [channelSlug]/page.tsx
│   │   └── settings/
│   │       ├── profile/page.tsx
│   │       ├── account/page.tsx
│   │       ├── notifications/page.tsx
│   │       └── billing/page.tsx
│   ├── learn/                        # Course player
│   │   └── [courseSlug]/[lessonSlug]/page.tsx
│   ├── cart/page.tsx
│   ├── checkout/
│   │   ├── page.tsx
│   │   ├── success/page.tsx
│   │   └── failed/page.tsx
│   ├── instructor/                  # Instructor portal
│   │   ├── layout.tsx
│   │   ├── dashboard/page.tsx
│   │   ├── courses/
│   │   │   ├── page.tsx              # Courses list
│   │   │   ├── new/page.tsx
│   │   │   └── [courseId]/
│   │   │       ├── edit/page.tsx      # Course builder
│   │   │       └── analytics/page.tsx
│   │   ├── analytics/page.tsx
│   │   └── earnings/page.tsx

```

```

├── students/page.tsx
├── org/                                # Organization portal
│   ├── [slug]/
│   │   ├── dashboard/page.tsx
│   │   ├── members/page.tsx
│   │   ├── courses/page.tsx
│   │   ├── analytics/page.tsx
│   │   └── settings/page.tsx
│   ├── admin/                          # Admin portal
│   │   ├── layout.tsx
│   │   ├── page.tsx                    # Admin dashboard
│   │   ├── users/page.tsx
│   │   ├── courses/page.tsx
│   │   ├── organizations/page.tsx
│   │   ├── payments/page.tsx
│   │   ├── moderation/page.tsx
│   │   ├── analytics/page.tsx
│   │   └── settings/page.tsx
│   └── moderator/page.tsx
├── components/
│   ├── ui/                             # shadcn/ui base components
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   ├── Topbar.tsx
│   │   ├── Footer.tsx
│   │   └── MobileNav.tsx
│   ├── courses/
│   │   ├── CourseCard.tsx
│   │   ├── CourseGrid.tsx
│   │   ├── CourseFilters.tsx
│   │   ├── CoursePlayer/
│   │   │   ├── VideoPlayer.tsx
│   │   │   ├── CurriculumSidebar.tsx
│   │   │   ├── LessonNotes.tsx
│   │   │   └── QAndA.tsx
│   │   └── builder/
│   │       ├── CourseBuilder.tsx
│   │       ├── ModuleCard.tsx
│   │       ├── LessonEditor.tsx
│   │       └── QuizBuilder.tsx
│   ├── quiz/
│   │   ├── QuizPlayer.tsx
│   │   └── QuizResult.tsx
│   ├── community/
│   │   ├── PostCard.tsx
│   │   ├── PostEditor.tsx
│   │   └── CommentThread.tsx
│   ├── notifications/
│   │   ├── NotificationBell.tsx
│   │   └── NotificationItem.tsx
│   └── shared/
│       └── FileUpload.tsx

```

```
|      ├── RichTextEditor.tsx
|      ├── StarRating.tsx
|      ├── ProgressBar.tsx
|      ├── Avatar.tsx
|      ├── Badge.tsx
|      ├── ConfirmDialog.tsx
|      └── EmptyState.tsx
├── features/                                # Feature-specific logic
|   ├── auth/
|   ├── courses/
|   ├── payments/
|   ├── notifications/
|   └── analytics/
├── hooks/
|   ├── useAuth.ts
|   ├── useCourse.ts
|   ├── useEnrollment.ts
|   ├── useProgress.ts
|   ├── useNotifications.ts
|   └── useDebounce.ts
├── lib/
|   ├── supabase/
|   |   ├── client.ts                      # createBrowserClient()
|   |   ├── server.ts                     # createServerClient()
|   |   └── middleware.ts
|   ├── api.ts                            # Edge Function callers
|   ├── utils.ts
|   ├── validations.ts                    # Zod schemas
|   └── constants.ts
├── store/                                  # Zustand stores
|   ├── authStore.ts
|   ├── cartStore.ts
|   ├── notificationStore.ts
|   └── uiStore.ts
├── types/
|   ├── database.ts                       # Generated from Supabase
|   └── app.ts                            # App-specific types
└── styles/
    └── globals.css
```

27. STATE MANAGEMENT — ZUSTAND STORES

27.1 authStore

- State: user, profile, roles, isLoading, isAuthenticated
- Actions: setUser, clearUser, updateProfile, hasRole(role)
- Initialized from supabase.auth.onAuthStateChange

27.2 cartStore

- State: items[{ courseId, price, discount }], coupon, total
- Actions: addItem, removeItem, applyCoupon, clearCart
- Persisted to localStorage

27.3 notificationStore

- State: notifications[], unreadCount
- Actions: addNotification, markRead, markAllRead
- Fed by Supabase Realtime channel

27.4 uiStore

- State: sidebarOpen, theme (light/dark), activeModal
- Actions: toggleSidebar, setTheme, openModal, closeModal

27.5 TanStack Query — Server State

- All remote data fetching via useQuery / useMutation hooks
- Keys: ['courses', filters], ['enrollment', courseId], ['profile', userId]
- Stale time: 5 min for course catalog, 0 for user-specific data
- Optimistic updates for likes, wishlist, lesson completion

28. SECURITY & RLS POLICIES

28.1 Key RLS Policies

profiles — SELECT	auth.uid() = id OR is_public = true
profiles — UPDATE	auth.uid() = id
courses — SELECT	status='published' AND visibility='public' OR (enrolled + org)
courses — INSERT/UPDATE	auth.uid() = instructor_id OR has role instructor/admin
lessons — SELECT	User enrolled in course OR lesson.is_free_preview
enrollments — SELECT	auth.uid() = user_id
enrollments — INSERT	auth.uid() = user_id
lesson_progress — ALL	auth.uid() = user_id
quiz_attempts — ALL	auth.uid() = user_id
payments — SELECT	auth.uid() = user_id OR has role admin
notifications — ALL	auth.uid() = user_id
admin tables	has role super_admin
moderation tables	has role moderator OR super_admin

28.2 Additional Security

- All Edge Functions validate JWT from Authorization header
- Selcom webhook: verify HMAC-SHA256 signature before processing
- Input validation with Zod on all forms and API inputs
- XSS prevention: Tiptap sanitizes HTML output
- Rate limiting: Supabase edge function rate limit + Cloudflare rules
- CORS: restrict to production domain
- File upload: validate MIME type server-side, not just extension
- SQL injection: 100% parameterized via Supabase client (never raw SQL in Edge Functions without parameterization)

29. PERFORMANCE & CACHING

Next.js ISR	Course catalog pages revalidated every 60s
Next.js Static	Homepage, about, pricing, public org pages
Image optimization	next/image with Cloudflare CDN
Lazy loading	React.lazy + Suspense for heavy components (editor, charts)
Virtual lists	react-virtual for long lists (notifications, lesson list)
DB indexes	courses(status, category_id, created_at), enrollments(user_id, course_id), lesson_progress(enrollment_id), notifications(user_id, is_read)
Pagination	All lists use cursor-based or offset pagination, max 25 per page
Supabase connection pooling	pgBouncer enabled on Supabase Pro
Video CDN	Bunny.net CDN handles global video delivery
Analytics queries	RPC stored procedures + materialized views for heavy aggregates

30. DEPLOYMENT & DEVOPS

30.1 Environments

Local	next dev + supabase local dev (Docker)
Staging	Vercel Preview deployments (auto on PR)
Production	Vercel main branch deploy

30.2 CI/CD Pipeline

- 69. Push to GitHub → GitHub Actions trigger
- 70. Run: TypeScript check, ESLint, unit tests
- 71. Vercel preview build and deploy
- 72. Supabase migrations run via Supabase CLI (supabase db push)
- 73. Merge to main → production deploy

30.3 Database Migrations

- All schema changes as SQL migration files in supabase/migrations/
- Version controlled in Git
- Applied with: supabase db push (staging) and supabase db push --linked (prod)
- Never modify Supabase schema directly in dashboard (non-reproducible)

31. PHASE ROADMAP — ALL 3 PHASES

Phase 1 — Foundation

Core platform — must be 100% functional and production-ready.

Authentication	Signup, login, Google OAuth, roles, onboarding
Public Website	Homepage, course catalog, course detail, about, contact
Course System	Full course builder, curriculum, video upload, articles
Quiz System	MCQ, T/F, auto-grading, retries
Assignment System	File upload, submission, manual grading
Enrollment	Free + paid enrollment, access control
Payments	Selcom Mobile Money + Card integration, webhooks, receipts
Student Portal	Dashboard, course player, progress tracking, notes
Instructor Portal	Course builder, analytics, earnings, student management
Certification	PDF generation, verification, download, share
Admin Portal	Users, courses, payments, configuration
Moderation	Course review queue, content reports
Email	Welcome, enrollment, payment, certificate emails
Notifications	In-app bell with real-time updates
Search	PostgreSQL full-text search
Responsive	Full mobile support

Phase 2 — Growth

Organization Portal	Full org management, invitations, private courses, org analytics
Community	Channels, posts, comments, Q&A, voting, mentions
Projects Showcase	Project creation, team management, public gallery
Programs & Events	Bootcamp registration, event management, attendance
Lab Resources	Digital makerspace with downloadable resources
Advanced Search	Typesense integration — typo-tolerant, faceted
Subscriptions	Monthly/annual plan subscriptions via Selcom recurring
Wishlists	Save courses for later
Reviews	Star ratings, written reviews, instructor replies
Coupons	Discount code system with usage limits
Blog / Articles	Platform blog for SEO and community

Push Notifications	Firestore Cloud Messaging for mobile/web push
Kits Catalog	Hardware kit listings with ordering workflow
PostHog Analytics	Full product analytics, funnels, retention
Multi-language	Swahili + English (i18n with next-intl)

Phase 3 — Innovation Ecosystem

AI Mentor Assistant	Claude/GPT-powered coding help, STEM guidance, project advice
IoT Device Integration	Arduino, ESP32, Raspberry Pi dashboard integration
Gamification	Points, badges, leaderboards, streaks, challenges
Live Sessions	Zoom/Whereby integration for live classes
Coding Exercises	In-browser code editor (Monaco) with test runner
Startup Incubation	Application system, mentor matching, milestone tracking
Competitions	Team formation, submission portal, judging system
Mentorship Program	Mentor profiles, booking, session tracking
Mobile App	React Native app with Expo (iOS + Android)
Advanced Certificates	Blockchain-verified credentials
API Marketplace	Org API for external integrations
White-label	Custom domain and branding for enterprise orgs

32. UX/UI DESIGN SYSTEM

32.1 Color Palette

Primary	#1E40AF (Blue 800) — CTAs, links, active states
Primary Light	#DBEAFE (Blue 100) — backgrounds, badges
Accent	#7C3AED (Violet 700) — innovation, projects
Success	#16A34A (Green 600) — completion, passed
Warning	#D97706 (Amber 600) — pending, expiry
Error	#DC2626 (Red 600) — errors, failed
Text Primary	#111827 (Gray 900)
Text Secondary	#6B7280 (Gray 500)
Background	#FFFFFF / #F9FAFB (Gray 50) for page backgrounds
Dark Mode	Full dark mode support via CSS variables + Tailwind dark:

32.2 Typography

Primary Font	Inter (all UI text)
Heading Font	Poppins or Satoshi (hero headings)
Code Font	JetBrains Mono (code blocks)
Base Size	16px (1rem) = Tailwind text-base
Scale	xs(12) sm(14) base(16) lg(18) xl(20) 2xl(24) 3xl(30) 4xl(36) 5xl(48)

32.3 Component Standards

Border radius	rounded-lg (8px) for cards, rounded-full for badges/avatars, rounded-md for inputs
Shadow	shadow-sm for cards at rest, shadow-md on hover
Transitions	transition-all duration-200 ease-in-out on interactive elements
Spacing	4px grid (Tailwind default: p-1=4px, p-4=16px, p-8=32px)
Card pattern	bg-white rounded-xl shadow-sm border border-gray-100 p-6 hover:shadow-md transition
Button primary	bg-blue-800 text-white px-6 py-3 rounded-lg font-semibold hover:bg-blue-900
Button secondary	border border-blue-800 text-blue-800 px-6 py-3 rounded-lg hover:bg-blue-50

Input	<code>border border-gray-300 rounded-lg px-4 py-3 focus:ring-2 focus:ring-blue-500 focus:border-transparent</code>
--------------	--

32.4 Layout Breakpoints

Mobile	< 768px — stack layout, bottom nav, condensed cards
Tablet	768px–1024px — 2-column grid, collapsible sidebar
Desktop	> 1024px — full layout, expanded sidebar, 3-4 column grids

33. IoT & DEVICE INTEGRATION (Phase 3)

33.1 Architecture

Devices send data to an MQTT broker (e.g. HiveMQ or Supabase Edge Function as HTTP endpoint). Data stored in `device_readings` table and displayed in learner's lab dashboard.

Supported devices	Arduino (HTTP), ESP32 (MQTT/HTTP), Raspberry Pi (Python SDK)
Protocol	HTTPS POST to <code>/functions/v1/iot/readings</code>
Payload	<code>{ device_id, sensor_type, value, unit, timestamp }</code>
Dashboard	Real-time charts via Supabase Realtime subscriptions
Authentication	Device API key per project (stored hashed in <code>device_registrations</code> table)

34. AI MENTOR ASSISTANT (Phase 3)

34.1 Features

- Floating chat widget available in course player and lab pages
- Powered by Claude API (`anthropic-sdk`)
- Context injected: current lesson content, course title, user's progress
- Capabilities: explain concepts, debug code, suggest resources, guide project development
- Conversation history stored per user per course (for context continuity)
- Conversation limit: 50 messages/day on free, unlimited on paid

34.2 Implementation

```
// Edge Function: /functions/v1/ai/chat
const response = await anthropic.messages.create({
  model: 'claude-opus-4-6',
  system: `You are UjuziLab AI Mentor. Student is learning: ${courseTitle}.`
    + `Current lesson: ${lessonTitle}. Help them understand and succeed.`,
  messages: conversationHistory,
  max_tokens: 1024
});
```

35. APPENDIX — ENVIRONMENT VARIABLES & CONFIG

35.1 Required .env.local Variables

NEXT_PUBLIC_SUPABASE_URL	Your Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon/public key
SUPABASE_SERVICE_ROLE_KEY	Edge Functions only — never expose to client
BUNNY_API_KEY	Bunny.net API key
BUNNY_LIBRARY_ID	Bunny.net video library ID
BUNNY_CDN_HOSTNAME	Your Bunny.net CDN hostname
SELCOM_API_KEY	Selcom merchant API key
SELCOM_API_SECRET	Selcom merchant API secret
SELCOM_VENDOR_ID	Selcom merchant vendor ID
SELCOM_WEBHOOK_SECRET	HMAC secret for webhook verification
RESEND_API_KEY	Resend email API key
RESEND_FROM_EMAIL	From email address (e.g. hello@ujuzihub.com)
FIREBASE_SERVER_KEY	Firebase Cloud Messaging server key
NEXT_PUBLIC_POSTHOG_KEY	PostHog project API key
NEXT_PUBLIC_POSTHOG_HOST	https://app.posthog.com
ANTHROPIC_API_KEY	Claude API key (Phase 3 AI mentor)
NEXT_PUBLIC_APP_URL	https://ujuzihub.com (production URL)
PLATFORM_FEE_RATE	0.30 (30% platform cut)
CERTIFICATE_NUMBER_PREFIX	UJZ

35.2 Supabase Database Triggers

on auth.users INSERT	→ INSERT into profiles (id, username from email prefix)
on enrollments INSERT	→ UPDATE courses.total_enrollments +1
on reviews INSERT/UPDATE	→ UPDATE courses.avg_rating, total_reviews
on lesson_progress UPDATE (status=completed)	→ recalculate enrollment.progress_percentage
on enrollment progress=100	→ call certificates/generate Edge Function
on courses UPDATE (status→published)	→ UPDATE courses.published_at = now()
on courses INSERT/UPDATE (title/description)	→ UPDATE search_vector = to_tsvector(...)

on payments UPDATE
(status→completed)

→ call enrollments/create Edge Function

35.3 Supabase RPC Functions (Stored Procedures)

<code>get_instructor_stats(instructor_id)</code>	Returns revenue, enrollments, avg_rating, completion_rate
<code>get_admin_kpis()</code>	Returns total_users, total_revenue_mtd, active_today, etc.
<code>get_course_analytics(course_id)</code>	Returns per-lesson completion rates, drop-off data
<code>search_all(query, limit)</code>	Full-text search across courses, instructors, projects
<code>get_recommended_courses(user_id)</code>	Returns courses based on enrolled categories
<code>recalculate_enrollment_progress(enrollment_id)</code>	Updates progress_percentage based on completed lessons

35.4 Key Third-Party Accounts to Create

74. Supabase — create project, enable Google OAuth, configure email templates
75. Bunny.net — create Stream library, get API key and CDN hostname
76. Selcom — merchant account, get API credentials, set webhook URL
77. Resend — create account, verify domain, create API key
78. Firebase — create project, enable FCM, download service account JSON
79. PostHog — create project, get API key
80. Vercel — connect GitHub repo, set environment variables
81. Cloudflare — add custom domain, enable proxy and DDoS protection
82. GitHub — monorepo setup, branch protection on main

TechStar UjuziLab — Complete Developer Blueprint

This document covers all 3 phases from ground up to market-ready product.

Built for React + Supabase + Vercel stack.